



FULL STACK QUEST

4ª Edição — Do Java Zero ao Full Stack, com Mapa Detalhado

EDIÇÃO COMPLETA · CADA TEMA EXPLICADO · PROGRESSÃO BÁSICO → AVANÇADO EM
TODOS OS MUNDOS

14

MUNDOS

47

FASES

14

BOSS FIGHTS

1

FINAL BOSS

Personagem: Desenvolvedor(a) Full Stack Java · Nível atual: Nível 1 (Aprendiz)



Como ler este mapa

Esta edição começa do zero absoluto em Java e vai até você colocar um sistema completo no ar. São **14 Mundos**, cada um fatiado em **Fases pequenas**. Cada Fase agora traz uma lista curta explicando o que cada tema realmente significa — não é mais só uma etiqueta solta. Além disso, dentro de cada Mundo as Fases sobem de nível: **Básico** → **Intermediário** → **Avançado**, para você sempre saber em que ponto da curva de dificuldade está.

☐ Fase de estudo ☐ Loot Secreto (curiosidade / aprofundamento) ☐ Boss Fight (projeto prático)

💡 DICA DE PROGRESSÃO

Os primeiros 4 Mundos são só Java puro, sem Spring, sem banco, sem web — a ideia é sair sabendo a linguagem de verdade antes de misturar frameworks. Só no **Mundo 5** (A Ponte para o Full Stack) o Java começa a conversar com banco de dados e com a internet. A partir daí, cada Mundo também evolui por dentro do Básico ao Avançado antes de te lançar no próximo. Vá com calma: uma Fase por sessão de estudo já é um ótimo ritmo.

TRILHA COMPLETA

- | | |
|--------------------------------------|---|
| 1 Fundamentos de Java | 8 Spring Boot Profissional |
| 2 O Coração do Java (POO) | 9 Segurança & Testes |
| 3 Estruturas & Exceções | 10 Fundamentos de Frontend |
| 4 Java Moderno & Concorrência | 11 TypeScript & React |
| 5 A Ponte para o Full Stack | 12 Arquitetura & Design Patterns |
| 6 Banco de Dados Relacional | 13 Docker & CI/CD |
| 7 A Web por Dentro (HTTP) | 14 Cache, Mensageria & Cloud |



A Vila Inicial (Fundamentos de Java)

+60 XP

Todo herói começa numa vila pacata aprendendo os golpes básicos. Aqui você aprende o vocabulário puro da linguagem Java, sem pressa — é a base de tudo que vem depois.

1.1 Sintaxe básica

BÁSICO

O alfabeto da linguagem: sem isso, nada mais faz sentido.

- ▶ **Variáveis e tipos primitivos** — como declarar e guardar números, textos e booleanos na memória.
- ▶ **Tipos wrapper** (Integer, Double...) — a versão "objeto" dos tipos primitivos, usada em coleções.
- ▶ **Operadores lógicos e matemáticos** — comparações (&&, ||, ==) e contas que formam qualquer regra de negócio.

1.2 Estruturas de controle

BÁSICO

É o que dá "decisão" e "repetição" ao seu código.

- ▶ **if / else e switch** — como o programa toma decisões diferentes dependendo da situação.
- ▶ **switch expressions** — a versão moderna e mais segura do switch tradicional, sem risco de "vazar" caso.
- ▶ **for, while, do-while** — as três formas de repetir um bloco de código várias vezes.

1.3 Strings e Arrays

BÁSICO

Como guardar e manipular várias informações de uma vez, sem banco de dados ainda.

- ▶ **Manipulação de Strings** — concatenar, cortar, comparar e formatar textos.
- ▶ **Arrays** — guardar várias posições de um mesmo tipo em uma única estrutura.
- ▶ **Iteração e busca** — percorrer um array ou string procurando ou processando valores.

💡 LOOT SECRETO

switch expressions: o Java moderno permite escrever switch como uma expressão que retorna valor direto (com - >), sem precisar de break em cada linha. É mais curto e evita um bug clássico: esquecer o break e "vazar" para o próximo caso sem querer.

🔪 BOSS FIGHT 1: A CALCULADORA DO APRENDIZ

Muito fácil

Missão: criar um programa de console com um menu de operações simples.

- Montar um menu (usando switch) com 4 operações matemáticas.
- Usar um while para o menu voltar a aparecer até o usuário escolher "sair".
- Guardar o histórico das últimas 5 contas em um Array e exibi-lo ao final.



O Coração do Java (POO)

+90 XP

Aqui mora a fundação de qualquer sistema robusto que você vai construir daqui pra frente — de um validador de CPF a um sistema completo de gestão.

2.1 Classes, objetos e encapsulamento

BÁSICO

A unidade básica de tudo em Java: como modelar "coisas do mundo real" em código.

- ▶ **Classes e Objetos** — como representar coisas do mundo real (Pessoa, Produto...) em código.
- ▶ **Construtores e this** — como inicializar um objeto e referenciar seus próprios atributos.
- ▶ **Encapsulamento** — esconder detalhes internos com `private` e liberar acesso controlado com Getters/Setters.

2.2 Herança e polimorfismo

INTERMEDIÁRIO

Como reaproveitar comportamento entre classes parecidas, sem repetir código.

- ▶ **Herança** — reaproveitar atributos e comportamentos de uma classe "mãe" em classes "filhas".
- ▶ **Classes abstratas** — classes que definem um contrato parcial, sem implementação completa.
- ▶ **@Override e super** — como sobrescrever um método herdado e ainda chamar a versão original.
- ▶ **Polimorfismo** — tratar objetos de tipos diferentes de forma única, através da classe mãe.

2.3 Interfaces e Records

INTERMEDIÁRIO

Interfaces são contratos; Records são a forma moderna de guardar dados imutáveis.

- ▶ **Interfaces** — contratos 100% abstratos que uma classe promete cumprir.
- ▶ **Estender x Implementar** — a diferença prática entre herdar de uma classe e implementar uma interface.
- ▶ **Records** — forma moderna e enxuta de criar classes imutáveis, ideais para representar dados simples.

💡 LOOT SECRETO

Herdar nem sempre é a resposta: antes de usar herança, pergunte "isso é um tipo daquilo, ou apenas TEM um daquilo?". Um Carro não é um Motor, ele TEM um motor. Errar essa pergunta é a causa nº 1 de hierarquias de herança confusas mais pra frente.

🗡️ BOSS FIGHT 2: O VALIDADOR DE CPF

Fácil

Missão: construir um pequeno sistema de cadastro de pessoas usando os 4 pilares da POO.

- Criar uma classe Pessoa com atributos privados e getters/setters.
- Criar Cliente e Funcionario herdando de Pessoa.
- Implementar uma interface Validavel com o método validarCpf().
- Criar um record simples PessoaResumo para exibir só nome e CPF formatado.



A Masmorra das Coleções

+90 XP

Sistemas de verdade lidam com muitos dados ao mesmo tempo e com coisas que dão errado. Aqui você aprende a guardar listas de informação na memória e a prever falhas antes que virem problema em produção.

3.1 Collections Framework

INTERMEDIÁRIO

Saber a diferença entre *List*, *Set* e *Map* é usado o tempo todo, em qualquer sistema.

- ▶ **List** (*ArrayList*/*LinkedList*) — coleções ordenadas que permitem elementos duplicados.
- ▶ **Set** (*HashSet*) — coleções que garantem que não existam elementos repetidos.
- ▶ **Map** (*HashMap*) — pares chave-valor, ideais para buscas rápidas por um identificador.

3.2 Tratamento de exceções

INTERMEDIÁRIO

Prever o que pode dar errado é o que separa um script de um sistema confiável.

- ▶ **try-catch-finally** — como capturar um erro em tempo de execução sem derrubar o sistema.
- ▶ **Checked x Unchecked** — a diferença entre exceções que o compilador obriga tratar e as que não.
- ▶ **Exceções personalizadas** — criar suas próprias classes de erro (ex: *ProdutoVencidoException*) para regras específicas.

3.3 Enums

INTERMEDIÁRIO

Muito melhor que usar texto solto para representar status ou categorias fixas.

- ▶ **Tipos enumerados** — representar um conjunto fixo de valores possíveis (ex: status de um pedido).
- ▶ **Enums com atributos e métodos** — enums podem carregar dados e comportamento, não só nomes.

💡 LOOT SECRETO

HashMap não tem ordem garantida: se você precisa que a ordem de inserção seja mantida, use *LinkedHashMap*; se precisa que fique sempre ordenado por chave, use *TreeMap*. Usar *HashMap* comum e depender da ordem é uma fonte clássica de bugs "aleatórios".

🗡️ BOSS FIGHT 3: O ESTOQUE QUE NÃO PERDOA

Fácil

Missão: montar um controle de estoque em memória (sem banco de dados ainda).

- Guardar os produtos em uma *List* e usar um *Map<String, Produto>* para busca rápida por código.
- Criar um Enum *StatusProduto* (*DISPONIVEL*, *VENCIDO*, *EM_FALTA*).
- Criar e lançar uma exceção personalizada *ProdutoVencidoException* ao tentar vender um item vencido.

📖 Se quiser, evolua essa ideia depois no seu *EstoqueSupermercado*.



O Grimório Moderno (Java Avançado)

+110 XP

Esses recursos diferenciam um código amador de um código profissional e otimizado. É a última parada de Java puro antes de partir para o mundo backend/web.

4.1 Lambdas e Streams API

AVANÇADO

Filtrar, mapear e reduzir coleções escrevendo bem menos código.

- ▶ **Expressões Lambda** — funções "anônimas" e curtas, passadas como parâmetro para outros métodos.
- ▶ **Stream: map, filter, reduce** — transformar, filtrar e resumir coleções de forma declarativa.
- ▶ **Collectors** — reunir o resultado de uma Stream de volta em uma lista, mapa ou valor único.

4.2 Optional & Date/Time API

AVANÇADO

Duas ferramentas que evitam os bugs mais clássicos e chatos de se debugar.

- ▶ **Optional** — representar explicitamente "pode não ter valor", evitando o NullPointerException.
- ▶ **LocalDate / LocalDateTime** — trabalhar com datas e horários de forma moderna e segura.
- ▶ **Fusos horários** — cuidados ao lidar com datas em sistemas usados em regiões diferentes.

4.3 Concorrência básica

AVANÇADO

Só o essencial: entender a ideia de múltiplas tarefas ao mesmo tempo.

- ▶ **Threads** — como o Java executa mais de uma tarefa "ao mesmo tempo".
- ▶ **Stack x Heap** — onde as variáveis realmente vivem na memória durante a execução.
- ▶ **Requisições simultâneas** — a ideia por trás de um servidor atender vários usuários ao mesmo tempo.

💡 LOOT SECRETO

Streams preguiçosas: operações como map e filter só executam de verdade quando você chama uma operação terminal, como collect ou forEach. Até lá, é só uma receita esperando para ser cozinhada.

🗡️ BOSS FIGHT 4: A REFATORAÇÃO DO GRIMÓRIO

Fácil/Médio

Missão: pegar o Estoque do Mundo 3 e deixá-lo com uma "cara mais moderna".

- Trocar pelo menos 3 loops for por Streams equivalentes (ex: filtrar produtos vencidos).
- Trocar uma busca que pode não encontrar nada para retornar Optional<Produto>.
- Usar LocalDate para a data de validade e calcular quantos dias faltam para vencer.



A Ponte para o Full Stack

+130 XP

Este é o momento em que você abandona interfaces gráficas embutidas (como Swing) e começa a fazer o Java conversar com a internet e com bancos de dados de verdade.



Maven/Gradle



JDBC/JPA



Spring Boot



APIs REST

5.1 Gerenciadores de dependência

BÁSICO

Como adicionar bibliotecas externas (como o próprio Spring) ao seu projeto.

- ▶ **Maven** — o gerenciador mais comum no ecossistema Spring, configurado pelo arquivo `pom.xml`.
- ▶ **Gradle** — alternativa ao Maven, mais flexível e usada em projetos maiores.
- ▶ **Bibliotecas externas** — como adicionar um framework sem precisar baixar nada manualmente.

5.2 JDBC x JPA/Hibernate

INTERMEDIÁRIO

Entender a conexão raiz com o banco antes de usar a "mágica" do ORM.

- ▶ **JDBC puro** — a conexão "raiz" do Java com o banco, escrevendo SQL na mão.
- ▶ **JPA/Hibernate** — um ORM que mapeia suas classes Java direto para tabelas do banco.
- ▶ **@Entity / @Table / @Id** — as anotações que ensinam o Hibernate a fazer esse mapeamento.

5.3 Spring Boot essencial

INTERMEDIÁRIO

O principal framework do mercado Java para construir sistemas web.

- ▶ **Inversão de Controle (IoC)** — o framework, e não você, decide quando criar e conectar os objetos.
- ▶ **Injeção de Dependência (DI)** — como uma classe "recebe" as dependências de que precisa, prontas para uso.
- ▶ **Beans e Component Scan** — como o Spring encontra e gerencia essas peças automaticamente.

5.4 APIs REST e DTO

AVANÇADO

Como seu Java vai finalmente "falar" com um frontend.

- ▶ **@RestController** — a porta de entrada que recebe requisições HTTP e devolve respostas.
- ▶ **GET/POST/PUT/DELETE** — os verbos usados para consultar, criar, atualizar e remover dados.
- ▶ **Respostas em JSON** — o formato universal que o frontend vai consumir.
- ▶ **Padrão DTO** — separar o que o banco guarda do que a API realmente expõe.

💡 LOOT SECRETO

A virada de chave: ao invés de o Java desenhar botões e janelas, ele passa a processar regras de negócio (calcular abastecimento, gerenciar estoque, administrar uma frota) e devolve só os dados puros — quem cuida do visual agora é a camada de frontend, em outro Mundo mais à frente.

🔪 BOSS FIGHT 5: O PRIMEIRO PORTAL

Médio

Missão: criar seu primeiro endpoint Java conectado a um banco de verdade.

- Criar um projeto Spring Boot via Maven com as dependências Web e JPA.
- Transformar sua classe Produto (do Mundo 3) em uma @Entity.
- Criar um @RestController com um endpoint GET que devolve a lista de produtos em JSON usando um DTO.



As Catacumbas de Dados

+120 XP

Todo bom sistema precisa de uma masmorra bem organizada para guardar seus tesouros. Aqui você sai do SQL mais simples até consultas avançadas, passando pela modelagem, e finaliza no PostgreSQL de verdade.

6.1 SQL básico

BÁSICO

O primeiro contato com a linguagem universal dos bancos relacionais.

- ▶ **SELECT** — como consultar e ler dados de uma tabela.
- ▶ **INSERT / UPDATE / DELETE** — como criar, alterar e remover registros.
- ▶ **WHERE e ORDER BY** — filtrar e ordenar os resultados de uma consulta.

6.2 Modelagem de dados

INTERMEDIÁRIO

Um bom modelo evita 80% dos problemas antes de escrever a primeira linha de código.

- ▶ **Entidade e Relacionamento** — como representar "coisas" e como elas se conectam entre si.
- ▶ **Cardinalidade** — definir se uma relação é um-para-um, um-para-muitos ou muitos-para-muitos.
- ▶ **Normalização** (1FN, 2FN, 3FN) — regras para evitar dados duplicados e inconsistentes.
- ▶ **Modelo ER** — o diagrama que você desenha antes de criar qualquer tabela.

6.3 SQL avançado

AVANÇADO

O que resolve perguntas reais de negócio, envolvendo mais de uma tabela.

- ▶ **JOINS** — combinar dados de duas ou mais tabelas em uma única consulta.
- ▶ **GROUP BY / HAVING** — agrupar registros e filtrar esses grupos (ex: total de vendas por cliente).
- ▶ **Subqueries e CTE** — consultas dentro de consultas, para problemas mais complexos.

6.4 PostgreSQL na prática

AVANÇADO

O banco relacional mais usado em projetos profissionais Java, colocado em ação.

- ▶ **Instalação e clientes** (DBBeaver/pgAdmin) — ferramentas visuais para explorar o banco.
- ▶ **Índices** — estruturas que aceleram buscas em colunas muito consultadas.
- ▶ **PRIMARY/FOREIGN KEY** — as regras que garantem a integridade dos relacionamentos.
- ▶ **Transactions** — garantir que um conjunto de operações aconteça tudo ou nada.

💡 LOOT SECRETO

Índice não é mágica grátis: ele acelera buscas (SELECT), mas deixa escritas (INSERT/UPDATE) um pouco mais lentas, porque o banco precisa atualizar o índice também. Por isso não se cria índice em tudo — só nas colunas realmente usadas em filtros e JOINS.

🗡️ BOSS FIGHT 6: A GRANDE MIGRAÇÃO

Médio

Missão: modelar e persistir de verdade o sistema do Mundo 5 em PostgreSQL.

- Desenhar um diagrama ER simples relacionando Produto, Fornecedor e Categoria.
- Criar as tabelas em PostgreSQL com chaves primárias e estrangeiras corretas.
- Escrever 3 consultas com JOIN para responder perguntas reais (ex: "produtos por fornecedor perto de vencer").



A Torre das Comunicações

+80 XP

Antes de deixar suas APIs mais robustas, vale entender de verdade a linguagem que os sistemas usam para se falar pela internet. Mundo curto, mas fundamental.

7.1 Como a web funciona

BÁSICO

Entender o caminho de uma requisição do navegador até o servidor.

- ▶ **HTTP / HTTPS** — o protocolo (e sua versão segura) usado por toda a internet.
- ▶ **DNS** — como um endereço como "google.com" vira um IP de verdade.
- ▶ **Cookies e Sessions** — como um site "lembra" de você entre uma requisição e outra.
- ▶ **CORS** — a regra de segurança que controla quem pode chamar sua API de outro domínio.

7.2 Verbos e status codes

INTERMEDIÁRIO

É o vocabulário que você já usou no Mundo 5 e vai usar todo dia como dev backend.

- ▶ **GET / POST / PUT / PATCH / DELETE** — a intenção por trás de cada tipo de requisição.
- ▶ **Códigos 2xx** — quando tudo deu certo.
- ▶ **Códigos 4xx** — quando o erro foi do lado de quem pediu (cliente).
- ▶ **Códigos 5xx** — quando o erro foi do lado do servidor.

7.3 JSON, REST e idempotência

AVANÇADO

O formato e o estilo de arquitetura usados por praticamente toda API moderna.

- ▶ **JSON** — o formato de dados usado para trafegar informação entre frontend e backend.
- ▶ **REST/RESTful** — o estilo de arquitetura mais usado para desenhar APIs.
- ▶ **Stateless** — a ideia de que o servidor não guarda memória de requisições anteriores.
- ▶ **Idempotência** — repetir a mesma requisição várias vezes deve ter o mesmo efeito de fazer uma vez.

💡 LOOT SECRETO

A visão de raio-x do navegador: abra o DevTools (F12) e vá na aba "Network". Toda vez que um site carrega, você vê as requisições reais indo e voltando, com headers, status code e o JSON de resposta — a melhor forma de estudar HTTP na prática.

🔪 BOSS FIGHT 7: CAÇADA DE REQUISIÇÕES

Muito fácil

Missão: virar um investigador de requisições HTTP (sem escrever backend nesta fase).

- Abra o DevTools em 2 sites diferentes e identifique 5 requisições GET e 2 POST.
- Anote pelo menos 3 status codes diferentes que você encontrar e o que cada um significa.
- Teste sua própria API do Mundo 5 no Postman/Insomnia e observe o JSON de resposta.



A Forja do Spring Boot

+150 XP

Agora que a ponte foi atravessada, é hora de deixar sua API com cara de profissional: camadas bem separadas, tratamento de erros e documentação de verdade.

8.1 Camadas e contratos da API

INTERMEDIÁRIO

Separar bem as responsabilidades é o que diferencia um projeto de faculdade de um profissional.

- ▶ **Controller → Service → Repository** — a separação clássica entre entrada, regra de negócio e acesso a dados.
- ▶ **DTOs de entrada e saída** — objetos específicos para o que a API recebe e devolve.

8.2 Validação e tratamento de erros

INTERMEDIÁRIO

Sua API precisa responder direito quando algo sai errado, não só quando dá certo.

- ▶ **Bean Validation (@Valid)** — validar automaticamente os dados recebidos (campo obrigatório, formato, etc.).
- ▶ **@ControllerAdvice** — centralizar o tratamento de erros de toda a API em um só lugar.
- ▶ **Exceções customizadas na Web** — transformar um erro de negócio em uma resposta HTTP clara.

8.3 Relacionamentos e paginação

AVANÇADO

Sistemas reais têm relacionamentos entre entidades e listas grandes demais para devolver de uma vez.

- ▶ **@OneToMany / @ManyToOne** — como representar relacionamentos entre entidades no JPA.
- ▶ **Lazy x Eager** — quando o Hibernate busca os dados relacionados: só quando pedir, ou sempre junto.
- ▶ **Paginação** — devolver os resultados em "páginas", em vez de uma lista gigante de uma vez.

8.4 Documentação de API

AVANÇADO

Uma API sem documentação é muito mais difícil de outros times (ou você mesmo) usarem.

- ▶ **OpenAPI** — o padrão que descreve formalmente os endpoints da sua API.
- ▶ **Swagger UI** — uma tela interativa gerada automaticamente para testar sua API.
- ▶ **Versionamento de API** — como evoluir uma API sem quebrar quem já a está usando.

💡 LOOT SECRETO

DTO não é frescura: expor a Entity direto na API parece mais rápido, mas vaza detalhes internos do banco e trava a evolução do sistema. O DTO é a "carta de embaixada" — só o que o mundo externo precisa ver.

🔪 BOSS FIGHT 8: A BIBLIOTECA ARCANA

Médio

Missão: construir uma API REST completa de biblioteca (Livros + Usuários + Empréstimos).

- CRUD completo (GET, POST, PUT, DELETE) para Livros e Usuários.
- Relacionar Empréstimo com Livro e Usuário, com paginação na listagem de livros.
- Documentar tudo com Swagger e testar via Postman/Insomnia.



A Fortaleza Blindada

+150 XP

Para que sua API suporte um frontend em produção, ela precisa ser blindada contra acessos indevidos e testável sem depender de clicar na tela toda hora.

Spring
Security

JWT



JUnit 5



Mockito

9.1 Autenticação básica

BÁSICO

Os dois conceitos que sustentam qualquer sistema de login.

- ▶ **Authentication x Authorization** — a diferença entre "provar quem você é" e "o que você pode fazer".
- ▶ **Hash de senha (BCrypt)** — nunca guardar senha em texto puro; o hash protege mesmo se o banco vazar.

9.2 Autorização e JWT

INTERMEDIÁRIO

O padrão mais comum para autenticar APIs stateless no mercado hoje.

- ▶ **Estrutura de um Token JWT** — como um token carrega a identidade do usuário de forma segura.
- ▶ **Filtros de segurança** — o "checkpoint" que valida o token antes de deixar a requisição passar.
- ▶ **Roles** ("admin", "user") — restringir endpoints de acordo com o perfil de quem está logado.

9.3 Testes automatizados

AVANÇADO

Garantir que a regra de negócio funciona sem reabrir o sistema a cada alteração.

- ▶ **JUnit** (Unit Test) — testar uma unidade de código isolada, de forma automática.
- ▶ **Mockito** (mocks) — simular dependências (como o banco) para testar só a lógica que importa.
- ▶ **MockMvc** — testar os endpoints da API sem precisar subir o servidor de verdade.

💡 LOOT SECRETO

O paradoxo dos mocks: com Mockito, você cria "dublês" para o banco de dados. Isso significa testar a lógica do seu Service sem nunca tocar no PostgreSQL de verdade — como treinar em uma sala de simulação antes da batalha real.

🔪 BOSS FIGHT 9: O SISTEMA DE CHAMADOS PROTEGIDO

Médio

Missão: transformar a Biblioteca do Mundo 8 (ou uma nova API de Helpdesk) em algo seguro e testado.

- Criar rota de login que devolve um Token JWT.
- Proteger endpoints: só ADMIN pode deletar registros, USER só pode criar/consultar.
- Escrever testes de unidade para o Service de criação (com Mockito).



O Ateliê das Ilusões (Frontend I)

+130 XP

De nada adianta um backend poderoso se ninguém consegue interagir com ele. Hora de sair do terminal e dominar o que o usuário realmente vê e toca, do HTML básico ao JavaScript assíncrono.

10.1 HTML semântico

BÁSICO

Estrutura correta facilita acessibilidade e deixa o CSS muito mais fácil depois.

- ▶ **Tags semânticas** — usar a tag certa (header, section, article) em vez de só <div>.
- ▶ **Forms e Inputs** — como coletar dados do usuário (texto, número, data, seleção).
- ▶ **Tables** — como estruturar dados tabulares corretamente.

10.2 CSS moderno

BÁSICO

A verdadeira "batalha" de todo frontend: alinhar as coisas do jeito certo.

- ▶ **Box Model** — como largura, altura, borda e espaçamento se combinam em cada elemento.
- ▶ **Flexbox** — o sistema mais usado para alinhar elementos em linha ou coluna.
- ▶ **Grid** — layouts bidimensionais, com linhas e colunas ao mesmo tempo.
- ▶ **Responsive Design** — adaptar o layout para celular, tablet e desktop.

10.3 JavaScript básico

INTERMEDIÁRIO

A linguagem que dá vida à página — considerada obrigatória para qualquer full stack.

- ▶ **let/const e arrow functions** — a forma moderna de declarar variáveis e funções.
- ▶ **Arrays e objects** — as estruturas de dados mais usadas no dia a dia do JS.
- ▶ **Condicionais e loops** — as mesmas ideias de Java, com uma sintaxe um pouco diferente.

10.4 JavaScript avançado

AVANÇADO

O que permite seu frontend conversar de verdade com uma API.

- ▶ **Promises / async-await** — como lidar com operações que demoram (como buscar dados de uma API).
- ▶ **DOM e Events** — como o JavaScript lê e altera a página, reagindo a cliques e ações.
- ▶ **Fetch API** — como fazer requisições HTTP direto do navegador, sem bibliotecas externas.

💡 LOOT SECRETO

Flexbox resolve 80% dos casos: antes de sair usando Grid para tudo, tente resolver o layout com Flexbox. O Grid entra quando você precisa de um layout bidimensional de verdade (linhas E colunas ao mesmo tempo).

🗡️ BOSS FIGHT 10: A PÁGINA VIVA

Fácil

Missão: criar uma página HTML/CSS/JS puro que consome a API do Mundo 8.

- Montar um layout responsivo simples com Flexbox (cards de livro).
- Usar `fetch()` para buscar os livros na sua própria API Spring Boot.
- Exibir os dados na tela dinamicamente, sem recarregar a página.



O Domínio das Interfaces (React)

+150 XP

Com o fundamento pronto, é hora de construir interfaces modernas e conectá-las ao backend blindado no Mundo 9.



TypeScript



React



Routing



State/Context

11.1 TypeScript essencial

BÁSICO

Vindo de Java, essa parte vai parecer familiar — é tipagem forte no mundo JS.

- ▶ **Types e Interfaces** — declarar explicitamente o formato dos dados, como em Java.
- ▶ **Generics** — escrever código reutilizável para múltiplos tipos, mantendo a segurança de tipo.
- ▶ **Enums** — o mesmo conceito de Java, agora no mundo do frontend.

11.2 React fundamentos

INTERMEDIÁRIO

A base de tudo: como React pensa em componentes e reatividade.

- ▶ **Components e Props** — blocos reutilizáveis de interface, que recebem dados de fora.
- ▶ **useState** — como um componente guarda e atualiza seu próprio estado.
- ▶ **useEffect** — como reagir a mudanças (ex: buscar dados quando o componente aparece na tela).
- ▶ **Forms controlados** — como capturar o que o usuário digita em tempo real.

11.3 React conectado ao backend

AVANÇADO

É aqui que o frontend finalmente conversa com a API do Mundo 9.

- ▶ **Routing** (React Router) — navegar entre "páginas" dentro de uma aplicação de página única.
- ▶ **Context API** — compartilhar dados (como o usuário logado) entre componentes sem repassar manualmente.
- ▶ **Rotas protegidas** — impedir o acesso a telas específicas sem estar autenticado.

💡 LOOT SECRETO

TypeScript é o "Java do front": ele adiciona tipagem forte ao JavaScript, com Interfaces e Generics parecidos com os que você já domina em Java. Evita bugs bizarros de "undefined não é uma função".

🔪 BOSS FIGHT 11: A INTERFACE DO CHAMADO

Médio

Missão: conectar um frontend em React + TypeScript ao backend protegido do Mundo 9.

- Tela de Login que envia dados para o Spring e guarda o Token JWT recebido.
- Dashboard organizado (Flexbox/Grid) listando os chamados vindos da API.
- Formulário que envia um POST para abrir um novo chamado.



O Santuário da Arquitetura

+100 XP

Aqui você começa a sair de desenvolvedor iniciante. Não é sobre aprender uma tecnologia nova, e sim sobre escrever código que qualquer outro dev consiga entender e manter.

12.1 SOLID na prática

BÁSICO

5 princípios simples que evitam boa parte dos problemas de manutenção de código.

- ▶ **Single Responsibility** — cada classe deve ter um, e só um, motivo para mudar.
- ▶ **Open/Closed** — código aberto para extensão, mas fechado para modificação direta.
- ▶ **Liskov, Interface Segregation, Dependency Inversion** — os três princípios restantes, focados em contratos bem desenhados.

12.2 Clean Code & Arquitetura em camadas

INTERMEDIÁRIO

Nomes bons e responsabilidades bem separadas valem mais que qualquer framework.

- ▶ **Clean Code** — nomes claros, funções pequenas e código que se lê como uma frase.
- ▶ **Clean Architecture** — organizar o sistema em círculos de dependência, do mais genérico ao mais específico.
- ▶ **MVC** — o padrão clássico que separa Model, View e Controller.

12.3 Design Patterns essenciais

AVANÇADO

Só os padrões mais usados no dia a dia — não vale tentar decorar os 23 clássicos.

- ▶ **Repository Pattern** — isolar o acesso a dados atrás de uma interface simples.
- ▶ **Factory** — centralizar a criação de objetos complexos em um só lugar.
- ▶ **Strategy** — trocar um algoritmo por outro em tempo de execução, sem if/else gigantes.
- ▶ **Builder** — construir objetos complexos passo a passo, de forma legível.

💡 LOOT SECRETO

Cuidado com a "padritite": não caia na armadilha de tentar encaixar um Design Pattern em absolutamente tudo. Um padrão só faz sentido quando resolve um problema real de duplicação ou acoplamento.

🔪 BOSS FIGHT 12: A GRANDE REFORMA

Médio

Missão: aplicar SOLID e um padrão de projeto em um sistema já existente seu.

- Escolher uma classe "faz-tudo" de um projeto seu e quebrá-la em responsabilidades menores.
- Aplicar o Repository Pattern separando o acesso a dados da regra de negócio.
- Documentar em um README o "antes e depois" da refatoração.

📖 Bom candidato: o [GerenciadorDeEscola](#) ou o [FrotaGro](#).



A Frota de Contêineres

+130 XP

Sua aplicação está pronta, mas só roda na sua máquina. Hora de empacotar tudo para que rode igual em qualquer lugar do mundo.



Docker

Docker
Compose

CI/CD



GitHub Actions

13.1 Docker essencial

BÁSICO

Isolar sua aplicação com tudo que ela precisa para rodar, em qualquer computador.

- ▶ **Image e Container** — a "receita" (imagem) e a "execução" (container) da sua aplicação.
- ▶ **Dockerfile** — o arquivo que descreve como montar sua imagem, passo a passo.
- ▶ **Volume** — como persistir dados fora do ciclo de vida do container.

13.2 Docker Compose

INTERMEDIÁRIO

Sobe backend, banco e frontend juntos com um único comando.

- ▶ **docker-compose.yml** — descrever vários serviços (backend, banco, frontend) em um único arquivo.
- ▶ **Serviços interligados** — como os containers conversam entre si pela rede interna do Compose.

13.3 Integração contínua

AVANÇADO

Automatizar build, teste e deploy a cada push, sem fazer nada manual.

- ▶ **GitHub Actions** — automatizar tarefas (build, teste, deploy) a cada push no repositório.
- ▶ **Pipeline: build → test → deploy** — a sequência clássica de uma esteira de entrega contínua.

💡 LOOT SECRETO

O contêiner imortal: o Docker isola sua aplicação com um mini sistema operacional próprio. Se rodou no container do seu PC, vai rodar *exatamente igual* na nuvem — acabou aquele clássico "na minha máquina funciona".

🔪 BOSS FIGHT 13: O SISTEMA EMPACOTADO

Médio

Missão: dockerizar o Sistema de Chamados construído nos Mundos 9 e 11.

- Dockerfile para o backend Spring Boot.
- docker-compose.yml subindo backend + PostgreSQL + frontend React juntos.
- Pipeline simples no GitHub Actions rodando os testes a cada push.



Os Reinos Distantes (Escala & Cloud)

+200 XP

O reino final. Aqui você conhece as ferramentas que fazem sistemas aguentarem muito uso ao mesmo tempo, e coloca tudo para rodar na nuvem.

14.1 Cache com Redis

BÁSICO

Evita bater no banco de dados toda vez, deixando o sistema muito mais rápido.

- ▶ **Key/Value** — a forma mais simples de guardar dados: uma chave aponta para um valor.
- ▶ **TTL** — definir por quanto tempo uma informação em cache continua válida.
- ▶ **Cache invalidation** — o desafio de saber quando um dado em cache ficou desatualizado.

14.2 Mensageria (RabbitMQ)

INTERMEDIÁRIO

O primeiro passo para aplicações distribuídas, processando coisas de forma assíncrona.

- ▶ **Producer / Consumer** — quem envia e quem processa uma mensagem, sem estarem diretamente conectados.
- ▶ **Queue** — a "fila de espera" onde as mensagens ficam até serem processadas.
- ▶ **Kafka** (visão geral) — uma alternativa voltada para grandes volumes de eventos.

14.3 Cloud com AWS

AVANÇADO

Não precisa aprender a AWS inteira — só o suficiente para colocar seu sistema no ar.

- ▶ **EC2** — servidores virtuais, onde sua aplicação realmente roda na nuvem.
- ▶ **S3** — armazenamento de arquivos (imagens, backups) na nuvem.
- ▶ **RDS** — bancos de dados relacionais gerenciados pela própria AWS.
- ▶ **IAM** — controle de quem (pessoas e serviços) pode acessar o quê dentro da conta AWS.

💡 LOOT SECRETO (BÔNUS: OBSERVABILIDADE)

Quando o sistema cresce, você passa a precisar de **Logs, Metrics e Tracing**. No ecossistema Java, o combo **Spring Boot Actuator + Prometheus + Grafana** é o caminho mais usado no mercado — vale conhecer depois de dominar o resto.

🐉 FINAL BOSS: O SISTEMA GLOBAL DE VENDAS

Desafiador

Missão: construir o projeto que resume toda a jornada, para mostrar em entrevistas.

- Spring Boot + PostgreSQL para Clientes, Produtos, Estoque e Pedidos.
- Redis fazendo cache das buscas de produtos.
- Frontend em React + TypeScript consumindo a API com autenticação JWT.
- TUDO orquestrado em Docker Compose, com pipeline de CI/CD configurada.



Equipe seu teclado mecânico e prepare as poções de energia (café).

Você começou lá no "Hello World" e chegou a um sistema completo em produção. A quest sempre continua — o resto agora é ganhar XP no mundo real.



Checklist de Progressão

Marque cada Boss Fight conforme for concluindo. Uma visão rápida de onde você está na jornada.

	MUNDO	BOSS FIGHT	XP
☐	1. Fundamentos de Java	A Calculadora do Aprendiz	+60
☐	2. Orientação a Objetos	O Validador de CPF	+90
☐	3. Estruturas & Exceções	O Estoque que Não Perdoa	+90
☐	4. Java Avançado	A Refatoração do Grimório	+110
☐	5. A Ponte para o Full Stack	O Primeiro Portal	+130
☐	6. Banco de Dados	A Grande Migração	+120
☐	7. HTTP & Web	Caçada de Requisições	+80
☐	8. Spring Boot Profissional	A Biblioteca Arcana	+150
☐	9. Segurança & Testes	O Sistema de Chamados Protegido	+150
☐	10. Frontend Fundamentos	A Página Viva	+130
☐	11. TypeScript & React	A Interface do Chamado	+150
☐	12. Arquitetura	A Grande Reforma	+100
☐	13. Docker & CI/CD	O Sistema Empacotado	+130
☐	14. Cloud & Escala	FINAL BOSS: Sistema Global de Vendas	+200

 LEMBRETE FINAL

Dominando até o fim do **Mundo 5** (A Ponte para o Full Stack), você já está pronto para começar a plugar tecnologias de frontend nas suas aplicações. Até o fim do **Mundo 9**, já tem uma base muito sólida para uma vaga de Java Backend Júnior. React, Cloud, mensageria e arquitetura avançada são o que te leva de Júnior para Pleno.